

Architecture and Tech Stack for the Crowdsourced Dataset Platform

Platform Overview and Requirements

Our platform is a **crowdsourced dataset marketplace** where organizations (companies, labs, universities) post dataset collection requests, and individual contributors sign up to provide data. Key requirements and characteristics include:

- **Moderate User Scale:** Initially hundreds to thousands of users, with potential to grow further. The architecture should handle current needs and be scalable for the future (horizontal scaling, load balancing) if the user base increases.
- **Rich Data Contributions:** Contributors will upload various file types – text, images, videos, audio, CSV/Excel, sensor signals, etc. The system must support large files and diverse formats efficiently.
- **Asynchronous Workflow:** Most interactions (browsing requests, uploading data, reviews) are asynchronous. However, some features (e.g. live progress updates, chat between requesters and contributors, or notifications) may require real-time capabilities.
- **Roles and Permissions:** Role-Based Access Control (RBAC) is needed. Typical roles include **Requester** (organization posting a dataset request), **Contributor** (providing data), and **Admin** (platform administrators). Permissions should ensure users only access authorized features (e.g. only requesters can approve/reject submissions, only admins can manage the platform) ¹.
- **Payments Integration:** The platform will handle payments via **Stripe**. Organizations pay contributors (and platform commission). We need secure payment processing, possibly holding funds and **payouts to multiple contributors** (a marketplace model). Stripe Connect is a likely solution, as it provides APIs to accept payments and **send payouts globally to third-party users** ².
- **Additional Services:** The platform will benefit from email notifications (for sign-ups, submission approvals, etc.), analytics for user behavior, cloud storage for files, and robust authentication.

With these requirements in mind, below is a detailed breakdown of the recommended software architecture – including front-end, back-end, database, storage, and supporting tools – with alternative options noted where appropriate.

Frontend Development

For the front-end, **Next.js (React)** is a highly suitable choice, and it aligns with your initial thought. Next.js is a React framework that offers multiple benefits for our use case:

- **Server-Side Rendering (SSR) and SEO:** Next.js can render pages on the server, which improves page load speed and makes content easily indexable by search engines ³. Our homepage and public dataset request listings will benefit from SSR for SEO (attracting contributors and organizations via search).
- **Static Site Generation (SSG):** Next.js can pre-build pages that don't change often. For example, public informational pages or open dataset pages could be statically generated for ultra-fast loads ³. This hybrid SSR/SSG capability gives us flexibility in serving content.

- **Built-in API Routes:** Next.js supports defining API endpoints within the same codebase (under `/api` routes) ³. This means we can implement backend logic (for simple cases) without a separate server, which is convenient for an MVP. Each API route is essentially a serverless function. For instance, a form submission or a request to fetch dataset info could be handled by a Next.js API route. This co-location of front-end and back-end code in one project simplifies development in early stages ⁴.
- **Developer Experience:** Using React with Next.js and TypeScript ensures a modern, robust front-end. We can leverage the React ecosystem (components, libraries, state management like Redux or React Context, etc.). Next.js also supports fast refresh for development, and a rich ecosystem of UI libraries (e.g. Material UI, Tailwind CSS integration) to speed up UI development.

Alternative Frontend Options: If not using Next.js, we could consider a standard React single-page app (SPA) created with tools like Create React App or Vite, or other frameworks like **Vue.js** (with Nuxt for SSR) or **Angular**. However, those would require a separate approach to SSR (or forego it). Given our need for SEO on dataset listings and the team's inclination towards Next.js, sticking with Next.js is ideal. It provides a nice balance of **fast initial development** (monolithic app structure) and the ability to scale into a more complex architecture later ⁴.

Frontend Structure: We will organize the front-end with clear page routes for main sections: e.g. Homepage, Browse Datasets, Dataset Request Detail (with description, guidelines, etc.), Contributor Dashboard (for contributors to see tasks they've joined), Requester Dashboard (manage their posted requests), Admin Panel, etc. Next.js allows dynamic routing for pages like `/requests/[id]` for dataset request detail pages. We'll also implement client-side dynamic interactions (such as form validations, progress bars on file uploads, etc.) using React components.

Additionally, we should ensure responsive design (mobile-friendly UI) and a clean UX for uploading contributions (perhaps a multi-step form guiding users through any required procedure).

Real-Time UI Updates: For “small features that could be real time,” on the front-end we can use web socket connections or polling to update data without full refresh. For example, if a requester is watching contributions come in, we might open a WebSocket to push new contributions in real-time. We can integrate a library like **Socket.IO** on the client to listen for updates from the server, or use Next.js built-in support for WebSocket (Next 13 has experimental `WebSocket` handling in server actions) if available. Alternatively, for a simpler approach, using an external service (like Pusher or Ably) can offload real-time messaging. The specific implementation will depend on how interactive these features need to be, but the front-end will be designed to handle real-time events (updating counts, notifications, chats) gracefully when needed.

Backend Development

The back-end is the core of our platform, handling business logic, user management, submissions, payments, etc. We have a few options, but a **Node.js backend** aligns well since we're using Next.js (JavaScript/TypeScript across the stack). Key considerations:

- **Node.js (JavaScript/TypeScript):** Using Node allows the same language (TypeScript) on front and back. We can start with Next.js API routes for simple needs, but as the project grows, it's wise to separate concerns for maintainability. We might implement a dedicated Node.js server (REST API or GraphQL) using a framework like **Express** or **NestJS**.
- **Express:** a minimalistic web framework, great for quickly setting up REST endpoints. We'd have full control over middleware, routing, etc.

- *NestJS*: a TypeScript framework that adds structure (Modular architecture, Dependency Injection, decorators). NestJS could be beneficial as complexity grows – for example, we can create modules for *Auth*, *Datasets*, *Contributions*, *Payments* etc., keeping the codebase organized. NestJS integrates well with databases and also supports WebSockets out-of-the-box if we need real-time features.

Why Node/JS? The team's familiarity with Next.js suggests strength in JS/TS. Node is event-driven and handles asynchronous I/O well – perfect for an app that will frequently interact with a database, cloud storage, and external APIs (Stripe) concurrently. The performance of Node will be sufficient for our scale (hundreds or thousands of users) and can scale horizontally by running multiple instances if needed.

- **Monolithic vs Microservices:** At the start, a **monolithic architecture** is recommended – i.e., a single unified codebase for the back-end (and possibly merged with the front-end if using Next API routes). This is **simple, fast to develop, and ideal for an MVP** ⁴ ⁵. All business logic (user auth, posting requests, file upload handling, payment processing) can reside in one application. Deployment is straightforward (one unit to deploy), and development is easier (no need to manage inter-service communication initially).

As the platform grows in features and traffic, we can gradually modularize or transition to microservices. For instance, we might spin off a separate microservice for heavy background processing or for the payment pipeline. In a **microservices architecture**, each service is independent (for example, a service just for file processing or an auth service). In the context of Next.js, the Next app often acts as **the frontend and an API gateway** that proxies or aggregates data from microservices ⁶. We don't need to start with microservices on day one – it adds operational complexity – but we design with an eye to the future. **It's an evolutionary journey:** start monolithic, perhaps introduce a modular monolith structure (group related logic into modules), and only **move to microservices when scale and team size demand it** ⁷.

- **Alternative Backends:** If the team has expertise in other languages, we could consider alternatives:
- *Python (Django / Flask / FastAPI)*: Python is common in AI fields. A Django backend could provide an all-in-one solution (ORM, admin panel, etc.), and FastAPI is great for building high-performance APIs. This might be considered if, say, we want to leverage Python for any AI-powered data validation or analysis within the platform. However, integrating a Python backend with a Next.js front-end would require maintaining separate projects and relying on REST/GraphQL interfaces. Given the cohesion of a full JS stack and the need to integrate tightly with Stripe and real-time features, sticking to Node.js is likely more efficient.
- *Serverless (BaaS) approach*: Another route is to use **Backend-as-a-Service** platforms to reduce custom backend development. For example, **Firebase** or **Supabase** can handle a lot of boilerplate: authentication, database, file storage (Firebase Storage or Supabase Storage), and even real-time listeners. Supabase in particular provides a PostgreSQL database with auto APIs and authentication. This could drastically speed up development for a small team. The trade-off is less flexibility (you adapt to their tools) and potential scaling cost, but it's an option to keep in mind for rapid prototyping. We could also use **serverless functions** (AWS Lambda, Vercel Serverless Functions, etc.) for specific tasks instead of a persistent server – for instance, Next.js API routes on Vercel are essentially serverless and auto-scale. A serverless architecture can scale well and offload a lot of server maintenance, but one must design carefully to avoid cold start latency and manage state (often using cloud databases and caching).

Backend Responsibilities: Regardless of implementation, the backend will handle: - **Authentication & Authorization:** Verifying users (login, registration) and enforcing roles on each endpoint. - **Dataset Request Management:** Endpoints to create a new dataset request (only requesters), listing requests

(public or filtered), retrieving details. - **Contribution Management:** Handling uploads (more on this in storage section), saving metadata about each contribution (which user, time, etc.), and marking status (e.g., pending approval, approved, rejected). - **Notifications & Real-time:** Possibly endpoints or WebSocket channels for notifications (like “your contribution was approved” or real-time chat between contributor and requester for clarifications). - **Payments Logic:** Coordinating with Stripe API to charge requesters and trigger payouts to contributors (or record credits for contributors). - **Admin Tools:** Endpoints for admins to manage users, moderate content, etc. - **External Integrations:** E.g., sending emails via an email service API, or analytics event tracking.

By using a robust framework (Next.js API routes or Express/NestJS), we can implement these with middleware for security (checking JWT or session for auth, role checks) and structure the code for maintainability (separating services, controllers, models in a typical Node project structure).

Real-Time Communication: If we need features like a **live chat or instant notifications**, we’ll incorporate WebSocket support in the backend. For example, if using NestJS, it has a WebSockets module that works well with Socket.IO. Alternatively, a smaller dedicated Node service using Socket.IO could be deployed for handling real-time events. Another approach is using a publish/subscribe mechanism – for instance, if a new contribution is submitted, the backend could publish an event to a message broker (like Redis Pub/Sub or a service like Pusher), and subscribers (the requester’s browser) get the update. We might start with simpler solutions (long-polling or periodic refresh) for MVP, but design the API to allow plugging in true push notifications as the platform matures.

Database Layer

For our database, a **relational database** is recommended, with **PostgreSQL** being a top choice. Key reasons for choosing a relational DB like Postgres:

- **Structured, Relational Data:** Our data has clear relationships – e.g., Users, DatasetRequests, Contributions, Payments, etc. PostgreSQL (or MySQL) excels at enforcing data integrity and handling complex joins or transactions (for example, ensuring a contribution and its payment record are saved atomically). If our application requires strong consistency and relationships, a relational DB is ideal ⁸.
- **ACID Compliance:** Postgres is fully ACID-compliant, meaning our transactions (like marking a contribution as approved and crediting a payout) will be handled reliably. This is important for financial transactions and record accuracy ⁹.
- **Complex Queries and Analytics:** We may need to query, for instance, “the number of contributions per user per request” or generate reports of contributions over time. PostgreSQL supports complex SQL, aggregate functions, and even full-text search capabilities if we want to allow searching dataset requests by keywords ¹⁰. This can be very useful (e.g., a search box for finding relevant dataset requests).
- **Scale and Familiarity:** Postgres can handle our expected scale (thousands of users, each potentially making many contributions). It’s a mature technology with a large community and many hosting options. We can start with a single Postgres instance or a managed service (like Amazon RDS, Google Cloud SQL, or services like Supabase) and later scale vertically or add read replicas. If needed, partitioning or sharding is possible, but unlikely required initially.

Alternative Database Options: - *MySQL/MariaDB:* Also relational, could be used similarly. Postgres often gets the edge for advanced features (JSON support, richer datatypes, extensions). MySQL would also work if the team is more comfortable with it, but we’ll assume Postgres for its features. - *NoSQL (MongoDB or DynamoDB):* A NoSQL database might be considered if we required very flexible schemas or massive horizontal scaling from the start. MongoDB stores JSON documents, which can be good for

rapidly changing or unstructured data. For example, if each dataset request had a completely arbitrary schema for contributions, a document store could adapt easily. Also, MongoDB can scale out on multiple servers more easily in some cases ¹¹. However, in our scenario, the core data is fairly structured (and we can accommodate variability by using JSON fields in Postgres if needed). We also benefit from SQL for complex queries. NoSQL sacrifices some relational querying and strict consistency, which we likely need for payments and clear linkages between tables. That said, certain parts of the platform could use NoSQL in a hybrid approach – e.g., using Mongo or a document store for logging user activity or storing schemaless metadata, while using Postgres for critical data ¹². But to keep things straightforward, a single primary database (Postgres) is recommended to start. - *NewSQL / Cloud Databases*: There are modern distributed SQL databases (CockroachDB, Google Spanner) which offer horizontal scaling with SQL interface. These might be overkill for now, but if global distribution or strong scaling became a requirement, these could be evaluated later.

Database Schema: We will design tables roughly as: - **Users:** (id, name, email, hashed_password, role, etc.) - **DatasetRequests:** (id, title, description, requester_id (FK to Users), status, reward per contribution, etc., plus fields describing requirements, maybe a JSON field for any structured instructions). - **Contributions:** (id, dataset_request_id (FK), contributor_id (FK to Users), submission_data (could be a file link or metadata JSON), timestamp, status (pending/approved/rejected), etc.) - **Payments/Transactions:** (id, contribution_id or dataset_request_id, amount, payer_id, payee_id, status, etc.) – to track money flow. - Possibly **Comments/Chat** table if we allow discussion on requests. - **Roles or Permissions:** Since roles are likely fixed (admin, contributor, requester), we might not need a separate table – just a role field on Users. But if we want a more granular permission system, we could have a Roles table and a join table mapping users to roles, or even a Permissions table. Initially, a simple enum field in Users for role is fine.

Using an **ORM (Object-Relational Mapper)** can speed up development. In a Node/TS environment, **Prisma** is a popular choice – it works with PostgreSQL and provides a type-safe API to the database. We define the schema in Prisma's schema language and it generates TypeScript client code. This ensures queries are correct and eases migrations. Other options include TypeORM or Sequelize. If using NestJS, TypeORM integrates nicely. Prisma is slightly more modern and developer-friendly with a Next.js stack.

Managed Database Service: To reduce DevOps burden, we can use a cloud-managed Postgres. For example, **AWS RDS (PostgreSQL)** if we host on AWS, or a service like Heroku Postgres, Supabase (which gives a Postgres and a nice interface), or ElephantSQL, etc. Managed DBs handle backups, updates, and scaling more easily. The Medium article on Next.js architecture notes that AWS RDS is a solid choice for a Next.js app needing SQL ¹³. We just need to ensure the connection is secure (using SSL, VPC if in cloud, etc.). In AWS, we could keep the database in a private subnet and have our backend in the same VPC for security ¹⁴.

File Storage and Handling Large Data

Handling user-contributed files is a major component of our platform. We expect potentially large files (videos, large image sets, datasets in CSVs, etc.). The strategy for file storage is crucial for scalability and cost. We will use **cloud object storage**, specifically **Amazon S3 (Simple Storage Service)** or an equivalent service (Google Cloud Storage, Azure Blob Storage, or even a service like Cloudinary for media, though Cloudinary is more for images/videos transformations).

Why S3? Amazon S3 is virtually unlimited storage, highly durable (99.999999999% durability), and can handle large files with parallel uploads. It's a standard choice for user-generated content storage. We can store files cheaply and serve them on demand. Importantly, S3 allows generating **pre-signed URLs**

for upload and download. This means the backend can hand the client a time-limited URL with credentials that allow the client to **upload directly to S3**, or download, without routing the data through our server.

Routing large uploads through our own backend would be a **bottleneck and a threat to stability** ¹⁵ . It can choke our server's bandwidth and processing if many users upload big files simultaneously. Instead, the backend will perform only an authentication and URL signing step, and the heavy file transfer goes directly to S3: 1. The contributor initiates an upload via the front-end. 2. Our backend API (authenticating the user) receives a request like "I want to upload file X for dataset Y". The backend checks the user is allowed, then calls S3 (using AWS SDK) to get a **pre-signed upload URL**. 3. The front-end uses this URL to **PUT** the file directly to S3. S3 handles the upload (we can even use multipart upload for very large files). 4. Once done, the front-end notifies our backend (or we could have S3 trigger a webhook) that the upload is complete, so we save a Contribution record in the database referencing the file's S3 key.

This approach *"offloads the entire data transfer process"* to S3, **simplifying our architecture and ensuring uploads are scalable and efficient without overloading our infrastructure** ¹⁵ .

We will also likely use S3 for storing any derived data or processed results. For example, if we generate a smaller thumbnail of an image, we can store it in S3 as well.

Serving Files and Downloads: For contributors and requesters to access the files, we have options: - If files are not meant to be public (e.g., dataset only accessible to the requester until fully compiled), we keep them private in S3 and use pre-signed URLs for download as well (backend gives a URL when an authorized user wants to download a file). - If we implement an **open repository of open-source datasets**, some files might be marked public. In that case, they could be placed in a public S3 bucket or behind a CDN. We could use **Amazon CloudFront** (a CDN) in front of S3 to globally cache and serve files quickly ¹⁶ . This is useful if large datasets will be downloaded by many users worldwide. - We'll set appropriate S3 bucket policies to ensure security (public access blocked unless intended, use server-side encryption for sensitive data, etc.).

Storage Alternatives: If not using AWS, Google Cloud Storage or Azure Blob have similar capabilities. Another interesting option is services like **Supabase Storage** or **Firebase Storage**, which under the hood use such object storage but provide integration with their auth (for example, Firebase Storage security rules can tie into Firebase Auth). Given our likely use of Stripe and possibly AWS services, using S3 directly is straightforward. If we were to go with a full serverless/BaaS approach, Firebase Storage could be considered, but it might be limiting for large files (Firebase has limits and complexity with large uploads too, although it can handle them with its SDK).

Handling Uploads on Frontend: We will implement a robust upload component on the front-end. Possibly using a library for resumable uploads (Tus protocol or something like AWS's multipart upload via SDK). At minimum, chunking large files in the browser can be done – this prevents the entire file from failing if the connection breaks, and allows showing progress. The Next.js app can use the AWS SDK in an API route or a backend endpoint to create multi-part upload URLs. Our front-end then uploads chunk by chunk. This is exactly the approach shown in a Next.js + S3 example which **chunks files and keeps the UI responsive** for large uploads ¹⁷ .

We should also consider file **validation**: restricting file types or sizes per the dataset request requirements. And scanning for malware (especially if we open the platform up, we don't want someone uploading a malicious file that others might download). We could integrate a virus scanning

step on the backend after upload (for instance, an AWS Lambda that triggers on S3 `ObjectCreated` events to scan the file, or an antivirus library if the file passes through the backend briefly). At least for now, enforcing size limits and type checking client-side and server-side is important.

Database vs Storage: We will **not** store large files in the SQL database (that would bloat it and degrade performance). We only store references (like the S3 object key or URL) in the Contributions table. This way, our DB holds metadata, while S3 holds the bulk data.

CDN & Caching: For frequently accessed content (like an image that appears on the site often, or a common open dataset file), using a CDN like CloudFront or Cloudflare can greatly reduce load times. We'll likely integrate a CDN once the public data grows. The Medium article suggests using CloudFront for assets to reduce latency ¹⁸ – the same applies to our user-uploaded content if it becomes heavy in traffic.

Authentication and Authorization

User Authentication: We need a secure system for users to register, log in, and manage sessions. Likely we will use **email/password authentication**, with the option to integrate social logins (Google, etc.) in the future for convenience.

- A straightforward approach in a Node/Next environment is using **NextAuth.js** (if we keep the backend integrated with Next). NextAuth can handle multiple providers and session management (it can issue secure cookies with session tokens or JWTs). It also makes it easy to integrate OAuth providers down the line.
- If we split out a custom backend (Express/Nest), we could use JWT authentication. For example, on login, the server issues a JWT signed with a secret, containing user ID and role, which the client stores (as an HTTP-only cookie or in localStorage depending on our setup) and sends on each request. The server then verifies JWT and attaches the user info to the request context. This is a common stateless auth approach that works well with REST APIs.
- Another route is using a managed auth service: **Auth0** or **AWS Cognito**. These services handle user credentials, password resets, multi-factor auth, etc., and provide JWTs or OAuth tokens to your app. The Medium case study on building a platform with Next.js suggests using AWS Cognito or Auth0 as options for secure authentication ¹⁹. The advantage is offloading the security concerns (they handle hashing, breach detection, etc.), but the downside is cost and some integration complexity. Given our scale, a self-implemented auth with good practices (bcrypt for hashing passwords, email verification step, rate limiting on login attempts) should suffice, but using Auth0/Cognito is not off the table if we want enterprise-level security from the start.

We'll also implement **email verification** on signup (send a link via email through our email service) to ensure valid accounts, and possibly support password reset via email.

Authorization (RBAC): As noted, we will implement Role-Based Access Control. The roles likely are: - **Admin:** full access to manage the platform (CRUD on all dataset requests, manage users, view stats). - **Requester (Client):** can create dataset requests, view and download contributions to their requests, accept/reject contributions, trigger payments. Cannot access other requesters' data. - **Contributor (Worker):** can browse available dataset requests, submit contributions, view their own contribution status and earnings. They shouldn't see details of others' contributions except maybe aggregate info or if data is open.

We enforce these both **in the UI** and **on the backend**: - In the UI, we'll hide or disable functionality not permitted (e.g., a contributor won't see admin dashboards). For example, we might have conditional rendering: if a user's role is not admin, they won't see the "Admin" menu, etc. This makes the UX clean and also hints at permissions. - However, **the backend is the ultimate gatekeeper**. Every API endpoint will verify the user's role before proceeding. For instance, an endpoint `POST /datasetRequests` will check that `req.user.role === 'requester'` (or admin) to allow creation. A `POST /contributions` will check the user is a contributor and maybe that they haven't exceeded any quota, etc. If not authorized, it returns 403 Forbidden.

We can implement RBAC checks manually or use libraries: - **Casbin or CASL**: These are libraries for Node.js that help define and enforce roles/permissions. For example, CASL can define abilities in a declarative way and be used on both frontend and backend. A Reddit discussion recommended CASL as straightforward for a JS stack ²⁰. - **Oso**: A newer library where you define policies in a declarative language (Polar) which can be more flexible for complex rules (like attribute-based access). Might be overkill for our initial simple roles.

Given our roles are few, a simple middleware function `checkRole(...allowedRoles)` can be written to protect routes (similar to the code shown in an RBAC implementation article ²¹). That article demonstrated a Node backend route protected by a `checkRole('admin')` middleware and React routes hidden based on roles ¹. We'll adopt that pattern.

User Roles Data Model: Likely just a field in the Users table (role = admin/requester/contributor). Possibly we also have to manage **organizations** if a requester account can have multiple members (e.g., a company might have a team). In future, we might add an Org table and allow multiple users with requester role under an organization, with shared billing. For now, we assume one user = one account.

Permissions and Data Scoping: We should ensure that even within a role, users can only access their own stuff. E.g., a requester should only fetch contributions that belong to their dataset request (use query filters with user's id) and a contributor should only see their own contributions (except if data is open to all). This will be handled via queries (always including a WHERE clause for the owner when appropriate) or additional backend checks.

Session Management and Security: We will use secure cookies for sessions if SSR pages need user context (NextAuth handles this by default with encrypted cookies), or rely on JWT in Authorization header for a pure API approach. We must enforce HTTPS so that tokens/cookies are not intercepted. Using HTTP-only cookies with SameSite settings can help mitigate XSS/CSRF issues if we go that route. Additionally, setting up rate limiting (e.g., using an Express middleware or a gateway) on auth endpoints and perhaps any critical operations is wise to prevent abuse.

Payment Processing with Stripe

Monetization is a key part of the platform. We've chosen **Stripe** as the payment processor, specifically Stripe Connect for marketplace functionality. Here's how we can integrate payments:

- **Stripe for Collecting Payments**: When an organization (requester) posts a new dataset request, we might require them to set a budget or pay an initial amount. There are a couple of models:
- **Per-Contribution Payments**: e.g., \$5 per valid contribution. In this model, we could charge the requester incrementally for each contribution that gets approved (or charge upfront for a certain number of contributions). The platform would hold these funds and then distribute to contributors.

- **Upfront Funding:** e.g., the requester funds \$500 into the system for the task, and as contributions come in, the funds are drawn down and paid out. If the task doesn't get enough contributions, maybe refunds or carry-over might be needed.
- **Posting Fee:** Additionally or alternatively, the platform might charge a posting fee or subscription for requesters to use the service.

Regardless of model, **Stripe Checkout** or **Payment Intents** can be used to charge the requester's credit card. We can integrate Stripe's API in our backend to create checkout sessions or payment intents when a requester is ready to pay. Stripe securely handles the card info on the frontend (via Stripe.js), so we don't touch raw card data (to stay out of PCI scope).

- **Stripe Connect for Payouts:** Stripe Connect is designed for marketplaces where you have a platform (us), connected accounts (contributors as payees), and customers (requesters paying). We will likely use **Stripe Connect Express accounts** for contributors:
- Each contributor, upon signing up or when they are ready to receive money, goes through a Stripe onboarding (providing their payment details, which could be bank account or debit card, plus any required identity verification for KYC/anti-fraud). Our platform creates a *Connected Account* in Stripe for that user ²².
- When a payment is made by a requester, the funds first go to our platform's Stripe account. Then we use Stripe Connect transfers or PaymentIntents with a connected account specified, to route funds to contributors. For example, if a contribution by Alice is approved and earns \$5, we can programmatically transfer \$5 (minus our commission) to Alice's Stripe connected account. Stripe will then payout to Alice's bank on a schedule (Stripe handles the actual bank transfer).
- Stripe's documentation provides a guide for this "collect then transfer" scenario ²³, analogous to a home-rental marketplace where a tenant pays and the platform pays out to the homeowner.
- We can configure our Connect integration such that our platform takes a fee (Stripe can automatically deduct a percentage cut for the platform).
- Using Stripe in this way ensures we remain compliant with financial regulations, since Stripe handles the money flow and KYC for us to a large extent.
- **Stripe Webhooks:** The backend will need to listen to webhook events from Stripe (like `payment_intent.succeeded`, `transfer.paid`, etc.) to know when to update our database. For instance, once a requester's payment succeeds, we mark the dataset request as funded. Or when a payout to a contributor is executed, mark that as paid. We must securely handle these webhooks (Stripe sends an HTTP POST to a webhook endpoint on our server with event details, we verify the signature and then process).
- **Security & PCI:** By using Stripe, we avoid storing any sensitive card information on our servers, which is important for security compliance. We will use Stripe's hosted components (Checkout or Elements) to handle card input. We should still follow security best practices (restrict who can trigger payouts – e.g., only our server should call Stripe APIs for payouts, not the client directly; validate amounts to prevent tampering, etc.).
- **Testing:** Stripe provides a test mode and dummy card numbers so we can simulate payments in development. We will make use of that to test various flows (successful payment, failed payment, etc.).
- **Alternative Payment Providers:** While Stripe is our choice (and widely used by marketplaces like Lyft, Instacart, Kickstarter, etc. as noted in the Sharetribe resource ²⁴), alternatives include PayPal's marketplace solution, Adyen, or Mangopay ²⁵. Stripe, however, is developer-friendly

and offers a comprehensive API that fits our needs, so we will stick to it unless there's a region it doesn't support (Stripe covers a lot of countries, but if we expand globally, we might need additional providers in some locales – something to monitor).

- **Compliance:** If this platform handles a lot of money, eventually we might need to consider tax reporting for payouts (Stripe can assist with generating 1099-K forms for US, etc.), and legal compliance in various jurisdictions. Stripe Connect simplifies much of this, but it's worth noting as a future consideration.

Infrastructure, Deployment, and DevOps

We will host the platform on a scalable, cloud-based infrastructure to ensure reliability and accommodate growth. Key elements of the deployment architecture include:

- **Hosting the Frontend (Next.js):**
 - A convenient option is deploying the Next.js app on **Vercel**, the cloud platform made by the creators of Next.js. Vercel excels at hosting Next.js apps – it automatically handles build, deployment, and scaling. We get automatic SSL, CDN edge caching for static assets/pages, and serverless function support for the API routes. A monolithic Next.js deployment on Vercel (with both our front-end and simple API routes together) could suffice initially ⁵, offering one-click deployments from our Git repository.
 - Alternatively, we can containerize the Next.js app (e.g., using Docker) and host on **AWS (EC2 or ECS), Heroku**, or **DigitalOcean App Platform**. If we separate backend services, we might use something like **Docker Compose** to orchestrate local development and ensure consistency in production. The dev.to article suggests Dockerizing microservices and using Compose for local setup ²⁶ ²⁷, which we might do once we break into multiple services.
 - If we go with AWS: We could use **AWS Amplify** or **AWS Elastic Beanstalk** for a somewhat managed deployment of our web app. Amplify can detect a Next.js app and deploy it, similar to Vercel. Elastic Beanstalk could host a Node app with SSR. Another modern approach is to use **AWS CDK** or CloudFormation scripts to set up infrastructure as code.
- In any case, we'll use a **CI/CD pipeline** (like GitHub Actions or GitLab CI) so that pushing code triggers tests and deployment to the hosting platform, ensuring quick iteration and rollbacks if needed.
- **Hosting the Backend/API:**
 - If we rely on Next.js API routes for the backend, Vercel would already cover that in a serverless fashion (with some limitations on execution time and memory). This is fine for many short-lived tasks like querying DB or calling Stripe. But long-running processes (like processing a large file) can time out on serverless.
 - If we split out a separate backend service (e.g., a NestJS API), we could deploy it on **Node.js server** hosted on Heroku, Render, or a container on AWS ECS/EKS. For simplicity, a service like **Render** or **Fly.io** can continuously deploy a Node app from Git and provide needed environment (they also provide managed PostgreSQL). Heroku is a classic choice for ease-of-use (though its free tier is gone, but for a small fee it provides a lot).
 - The backend should be behind SSL (if separate from Vercel, we'll ensure to configure HTTPS, or use a proxy).
- **Scaling backend:** Initially, one instance can handle our traffic. As usage grows, we can run multiple instances behind a load balancer (most PaaS do this for us or make it easy – e.g.,

Heroku auto-scales dynos, AWS ECS can scale tasks). We should design stateless servers (don't store user session in memory; instead use JWT or a shared session store like Redis) so that scaling out is easy.

- **Database:** If using a managed DB (like RDS or Supabase), that service will handle replication, backups, etc. If we self-host the DB (not recommended early on), we'd deploy it on an EC2 or Docker container and need to manage durability (which adds DevOps overhead). So likely, we'll go managed. For example, if deploying all on AWS, an **RDS Postgres instance** in a single AZ with read replicas in others (for failover) could be set up. We'll keep credentials safe (using environment variables or a secret manager).
- We must also plan for **database migrations** (Prisma or TypeORM migrations) and have a strategy for migrating schema as the product evolves (running migrations as part of CI/CD or with a manual admin command).
- **Object Storage:** If AWS S3 is used, it's a fully managed service – we just need the backend to have credentials (AWS access key/secret) with permissions to read/write our specific S3 bucket. On Vercel or other platforms, we'll set these as environment variables. We might create a separate IAM user in AWS with restricted S3 access to use from our app.
- For serving content globally, if using CloudFront in front of S3, that's also managed. We'll configure a CloudFront distribution with our S3 as origin. This can be set up later when needed, but is relatively straightforward.
- **Scaling and High Availability:** The architecture described is mostly horizontally scalable. If user load increases, we can:
 - Run more instances of the backend API (behind a load balancer).
 - Enable auto-scaling on Vercel (they do this automatically for serverless functions) or on whatever container platform we use.
 - Scale the database (move to a larger instance, or add read replicas if needed for read-heavy workload).
 - Leverage caching: e.g., we can add a **Redis cache** for frequent queries or to store session data if we need quick lookups. Redis could also be used for background job queues.
 - Use a job queue system: For any heavy background tasks (e.g., generating a report, sending bulk emails, or video transcoding if we allowed that), it's wise to not block web request threads. We can use something like **BullMQ (Redis-based queue)** or a cloud queue (AWS SQS) plus workers. For example, upon a new file upload, we enqueue a job to do post-processing (if any), and a separate worker process (could be another Node service or a serverless function triggered by S3 event) handles it asynchronously. This keeps the web server responsive. The serverless architecture note we found mentions offloading asynchronous tasks (like sending emails or processing images) to serverless functions for better scalability ²⁸ – that's exactly the idea here. We could have, say, an AWS Lambda that sends emails or processes images triggered by queue or S3 events.
- **Monitoring & Logging:** As a serious platform, we need good monitoring:
 - We will implement logging of important events and errors. If using Node, libraries like Winston or Bunyan can log to console (which in cloud goes to a log service, e.g., CloudWatch Logs on AWS

or Vercel's logs). We might integrate a log management tool if needed (Datadog, Loggly, etc., or even an ELK stack if self-managed).

- **Error tracking:** Using a service like **Sentry** will allow us to catch exceptions in both frontend and backend with stack traces, helping debug issues in production.
- **Performance monitoring:** We can use Google Analytics to see front-end page load metrics, and APM (Application Performance Monitoring) tools like NewRelic or Datadog APM for server performance if needed in future (to catch slow DB queries, etc.).
- **Analytics:** On the business side, beyond Google Analytics for page views, we might want to track user actions (like how many contributions per request, conversion funnel from visiting a request to contributing, etc.). Tools like **Mixpanel** or **Segment** can help capture such events for analysis. Initially, we can instrument some custom tracking and analyze manually or with simpler tools.
- **Security:** We will deploy with security best practices:
 - Use **HTTPS** everywhere (Vercel and most cloud platforms provide TLS termination).
 - Keep all secrets (DB passwords, Stripe API keys, JWT secret, etc.) out of code and in secure environment configurations. Rotate keys if needed.
 - Possibly use a Web Application Firewall (WAF) if on AWS (AWS WAF) or via Cloudflare in front to protect against common attacks.
 - Regularly update dependencies to patch vulnerabilities.
 - Implement server-side validation on all inputs (even if we have client-side validation) to prevent things like SQL injection or malformed data (ORM helps here).
 - Rate limit critical endpoints to prevent abuse (like someone spamming file uploads or payment attempts).
 - Backups: ensure the database has automated backups (most managed DBs do nightly backups at least). For S3, we could enable versioning or at least have a backup plan for critical data if needed (though if things are truly crowdsourced and open, permanent loss might not be catastrophic, but we should try to avoid any data loss especially for things like payment records).

Additional Tools and Services

Beyond the core components, a few supplementary services and frameworks will improve our platform's functionality and our development speed:

- **Email Service:** We will integrate an email delivery service to send out emails for account verification, password resets, notifications (like "Your contribution was accepted!" or weekly digests perhaps). Popular choices are **SendGrid**, **Mailgun**, **Amazon SES**, or even transactional email services like **Postmark**.
- SendGrid offers a free tier for small volumes and is easy to use via Node (SMTP or API). We can use it to send templated emails (e.g., using Handlebars templates for consistent branding).
- We should send emails asynchronously (don't block user action waiting for email send; use a background job or at least do not await the API call in the request/response cycle).
- Ensure to verify domain, set up SPF/DKIM for deliverability.
- **Analytics and User Tracking:** As mentioned, Google Analytics will be added to the front-end to track page visits and marketing metrics. If we want to deeply analyze user behavior within the app (like which requests are most viewed, where users drop off in a contribution process), we might embed an analytics SDK (Mixpanel or Amplitude). This can wait until we have enough usage to analyze, but planning for it is good (just ensure our privacy policy covers tracking).

- **Search Functionality:** As the number of dataset requests grows, we need a good search/filter on the front-end. In early stages, we can rely on database queries (with indexes on keywords or using Postgres Full Text Search) ¹⁰ to implement a search bar. For more advanced capabilities (fuzzy matching, autosuggestions), an external search engine like **Elasticsearch** or **MeiliSearch** could be integrated. This would be a separate service syncing data from the DB. We likely won't need this immediately, but it's on the radar as a future enhancement once we have dozens or hundreds of requests to sift through.
- **Testing and QA:** We'll use frameworks like **Jest** (for unit tests in Node/Next), and perhaps integration testing tools (like Cypress or Playwright) for end-to-end testing of critical flows (user registers, posts a request, contributor submits data, etc.). This ensures our architecture choices (especially the integration of all these services) work together as expected. With Next.js, we can also test API routes as simple functions.
- **Documentation:** For internal or external use, if we expose an API to third parties (maybe in future allowing programmatic submission of data or retrieval), we'd document it (Swagger/ OpenAPI documentation for REST, or GraphQL schema docs if we go that route).
- **Automation and Admin Tools:** It's helpful to have an admin interface to manage the platform. We could quickly scaffold something using open-source admin tools (like **Strapi**, or even Django admin if we had a Django backend, or an auto-generated admin from our data models). But since we are building custom, perhaps a simple admin section in our frontend for admins, with a set of API endpoints to fetch user lists, flag inappropriate contributions, etc. Logging important actions (audit log) is useful for debugging issues (like who changed a payment status, etc.).
- **Potential Integrations:** Down the line, we could integrate with partner services:
 - University SSO or similar if partnering with universities (for easy login).
 - Cloud compute services if we ever allow running some processing on the contributed data (maybe out of scope, but e.g., if validating images, using AWS Rekognition or similar AI service).
 - Community forums or chat (maybe integrate a forum software or Slack/Discord for the community of contributors).
 - But these are optional; initially focus is core platform.

Scalability and Future-Proofing

To summarize our approach to scalability: - **Start Simple, Stay Flexible:** We begin with a monolithic Next.js application (frontend + basic API routes) for speed ⁴. As requirements expand, we modularize the code (group by feature) and possibly extract microservices for distinct domains (like a separate service purely for handling the heavy file conversion tasks, or a dedicated payments service to isolate financial logic). We recognize that *"microservices shine when priorities are scale and independent deployment, but they introduce complexity"*, so we will only adopt them when the **time and team are right** ²⁹. - **Cloud-Native Readiness:** All components are chosen with cloud scaling in mind. The stateless nature of our services (with externalized state in DB, storage, etc.) means we can scale horizontally. Using managed services (DB, storage, Stripe) offloads a lot of scaling burden to cloud providers. - **Monitoring Performance:** We will keep an eye on performance metrics. If we see database load climbing, we can add indexes or caching. If the Node server CPU is high, we scale out or optimize code. If file downloads become huge, we ensure our CDN is properly caching and maybe consider peer-to-peer distribution for open datasets (just a thought if data sets become extremely large, though likely not needed if budgets are per contributor rather than massive single files). - **Security & Updates:**

Staying updated with security patches for dependencies and regularly reviewing our security (especially around payments and file uploads) will ensure the platform remains robust as it grows.

Finally, it's worth noting that the technology choices made (Next.js, Node, Postgres, S3, Stripe, etc.) are all well-established and have a rich ecosystem. This means as our project grows, we'll find libraries and community support to extend functionality. For example, implementing two-factor auth, or advanced auditing, or scaling out with a microservices pattern – all have known solutions in this stack.

By carefully combining these tools and services, we set up a platform that is **comprehensive and robust** yet can be built by a small team. We leverage proven frameworks to avoid reinventing the wheel (for auth, payments, file storage), allowing us to focus on the unique features of our platform (facilitating data collection collaboration).

Below is a summary of the recommended tech stack and architecture for clarity:

- **Frontend:** Next.js (React & TypeScript) – SSR for SEO, SPA-like interactivity, hosted on Vercel (for fast global access and ease of deployment).
- **Backend:** Node.js with Express/NestJS (TypeScript) – initially as part of Next.js API routes or a combined app, later possibly separate service. Implements RESTful APIs (or GraphQL if needed in future) to handle core logic. Deployed on a scalable platform (Vercel serverless, or Node server on AWS/Heroku/Render as needed).
- **Database:** PostgreSQL – relational database for structured data (users, requests, contributions, transactions). Use a managed service (AWS RDS or Supabase, etc.) ¹³. Access via an ORM (Prisma/TypeORM) for productivity.
- **File Storage:** AWS S3 – for all user-uploaded files, with backend-managed pre-signed URLs for direct uploads ¹⁵. Use CloudFront CDN in front of S3 for efficient delivery of content to users worldwide ¹⁶. Ensure large file support with multipart uploads.
- **Authentication:** NextAuth (if using Next's built-in backend) or JWT-based auth in Node. Consider Auth0/Cognito if external auth service is desired ¹⁹. Implement robust RBAC checks in both frontend and backend to enforce permissions ¹.
- **Payments:** Stripe – use Stripe Checkout/Payments for charging requesters, and **Stripe Connect** for marketplace payouts to contributors ². Handle Stripe webhooks for payment events.
- **Services & Integrations:** Email notifications via SendGrid (or similar), Google Analytics for basic tracking, possibly Mixpanel for product analytics. Use a job queue or serverless functions for background tasks (email sending, image processing) to keep the system responsive ²⁸.
- **DevOps:** Version control with Git (GitHub or GitLab) and CI/CD for automated testing and deployment. Docker for containerization if needed. Infrastructure as code (CloudFormation/Terraform) for reproducibility if we go deep into AWS infrastructure. Monitoring with Sentry (for error tracking) and platform logs for debugging.

With this architecture, our team can confidently start building the platform. It balances **development speed** (using high-level frameworks and managed services) with **future scalability** (cloud-native components and modular design). Each piece of technology was chosen to fit our specific needs of a data crowdsourcing platform, ensuring we can handle the unique challenges of large file management, multi-party payments, and user collaboration effectively.

Overall, this setup provides a strong foundation to turn our startup idea into a robust product, while remaining adaptable as we learn more about our scale and usage patterns.

Sources:

- Next.js monolithic architecture – ideal for fast development and unified deployment ⁴ ⁵ . Next.js features (SSR, SSG, API routes) make it a strong choice for our platform ³ .
- Monolith vs. microservices – start simple and evolve when needed; microservices allow independent scaling but add complexity ³⁰ ²⁹ .
- PostgreSQL vs NoSQL – choose Postgres for relational integrity and complex queries; NoSQL (Mongo) only if flexible schema or massive scale is required ⁸ ¹¹ .
- PostgreSQL supports advanced features like full-text search which we can utilize for dataset search functionality ³¹ .
- Uploading large files – use S3 with pre-signed URLs to offload file transfer from our servers, ensuring scalable and efficient uploads ¹⁵ .
- AWS services complement Next.js – use S3 & CloudFront for static assets and uploads, AWS RDS for Postgres, and potentially Lambda for serverless functions ¹⁶ ¹³ .
- Role-Based Access Control is essential to keep the app secure and well-structured, ensuring users only access what they should ¹ .
- Stripe Connect is a proven solution for marketplace payments, allowing us to accept payments and then pay out to third-party users (contributors) while handling compliance ³² .

¹ ²¹ Implementing Role-Based Access Control (RBAC) in Node.js and React | by Frontend Highlights | Medium

<https://medium.com/@ignatovich.dm/implementing-role-based-access-control-rbac-in-node-js-and-react-c3d89af6f945>

² ²⁴ ²⁵ ³² Stripe Connect marketplace payments: overview - Sharetribe

<https://www.sharetribe.com/academy/marketplace-payments/stripe-connect-overview/>

³ ¹³ ¹⁴ ¹⁶ ¹⁸ ¹⁹ Exploring Modern Web App Architectures with NextJS and AWS | by Ankur Sharma | Medium

<https://medium.com/@developeranku/exploring-modern-web-app-architectures-with-nextjs-and-aws-fa44407f90d9>

⁴ ⁵ ⁶ ⁷ ²⁸ ²⁹ The Ultimate Guide to Software Architecture in Next.js: From Monolith to Microservices - DEV Community

https://dev.to/shayan_saed/the-ultimate-guide-to-software-architecture-in-nextjs-from-monolith-to-microservices-i2c

⁸ ⁹ ¹⁰ ¹¹ ¹² ³¹ PostgreSQL vs. MongoDB: Differences, Strengths, and Use Cases | Estuary

<https://estuary.dev/blog/postgresql-vs-mongodb/>

¹⁵ ¹⁷ The Architecture That Lets Us Sleep: Scalable Uploads with S3 presigned URLs - DEV Community

<https://dev.to/oliverke/the-architecture-that-lets-us-sleep-scalable-uploads-with-s3-presigned-urls-1jf3>

²⁰ Any RBAC/ACL libraries? : r/node - Reddit

https://www.reddit.com/r/node/comments/1c9s1au/any_rbacacl_libraries/

²² ²³ Checkout

<https://docs.stripe.com/connect/collect-then-transfer-guide>

²⁶ ²⁷ ³⁰ Building Scalable Microservice Architecture in Next.js - DEV Community

<https://dev.to/hamzakhan/building-scalable-microservice-architecture-in-nextjs-1p21>